3C

DESARROLLO DE SOFTWARE

Desarrollo de aplicaciones de Software para la solución de problemas

Saber, valorar, elegir

PADRE ARRUPE
Colegio Español

DOCENTE: Daniel Sosa

COLEGIO ESPAÑOL PADRE ARRUPE

Colegio internacional.

Tercer Año de Bachillerato de Software

Desarrollo de aplicaciones de Software para la solución de problemas

Periodo 1.

GUÍA 9 – JDBC con JavaFX

OBJETIVOS DE LA PRÁCTICA

Objetivo General

Desarrollar una aplicación de escritorio utilizando JavaFX y JDBC que permita gestionar productos en una base de datos relacional mediante operaciones CRUD, aplicando una arquitectura organizada y buenas prácticas de programación.

Objetivos Específicos

1. Comprender la estructura y funcionamiento de JavaFX como framework para el desarrollo de interfaces gráficas.
2. Implementar la conexión a una base de datos MySQL utilizando la API JDBC.
3. Aplicar sentencias SQL para realizar operaciones de inserción, consulta y eliminación de registros.
4. Integrar la interfaz gráfica con la capa de acceso a datos mediante el patrón MVC.
5. Utilizar FXML para separar la vista de la lógica de negocio.
6. Implementar buenas prácticas de programación como el uso de PreparedStatement para prevenir vulnerabilidades de seguridad.
7. Organizar el proyecto en paquetes estructurados que favorezcan la mantenibilidad del código.
8. Analizar el flujo completo de una aplicación cliente–servidor en un entorno de escritorio.

INTRODUCCIÓN TEÓRICA

En el desarrollo de aplicaciones modernas, la interacción entre interfaces gráficas y bases de datos es un componente fundamental en sistemas empresariales. Las aplicaciones de escritorio continúan siendo relevantes en entornos administrativos, comerciales y educativos donde se requiere estabilidad, control de recursos y funcionamiento offline.

JavaFX es una plataforma para la construcción de interfaces gráficas en Java que permite diseñar aplicaciones con una arquitectura basada en componentes, separación entre vista y lógica mediante FXML y soporte para estilos mediante CSS. A diferencia de tecnologías anteriores como Swing, JavaFX ofrece una estructura más moderna, orientada a propiedades y enlazamiento de datos (binding), lo que facilita el desarrollo de interfaces dinámicas.

Por otra parte, JDBC (Java Database Connectivity) es la API estándar de Java para la conexión y manipulación de bases de datos relacionales. Permite ejecutar sentencias SQL, gestionar conexiones, manejar resultados y aplicar buenas prácticas como el uso de PreparedStatement para evitar vulnerabilidades como la inyección SQL.

La integración de JavaFX con JDBC permite desarrollar aplicaciones de escritorio conectadas a bases de datos, implementando operaciones CRUD (Create, Read, Update, Delete) bajo una arquitectura organizada, comúnmente basada en el patrón Modelo–Vista–Controlador (MVC). Esta práctica permite comprender cómo interactúan las capas de presentación, lógica y persistencia dentro de un sistema informático.

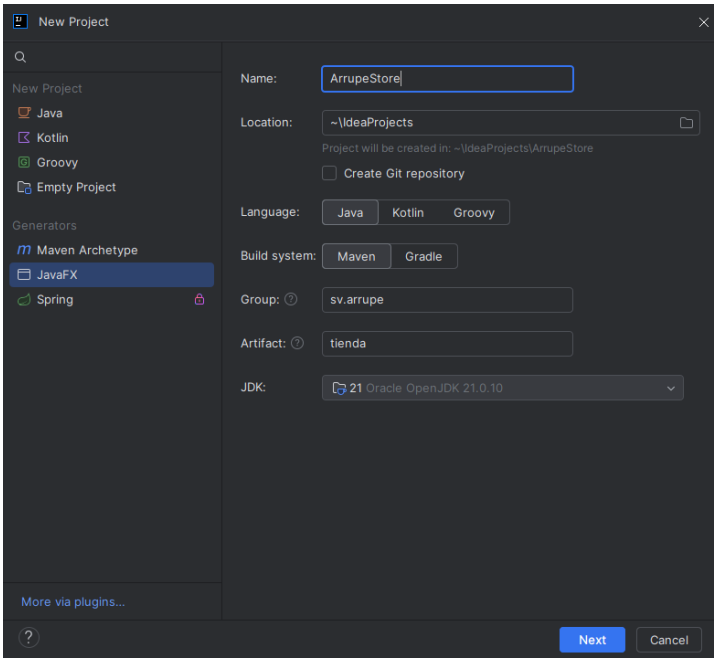
La aplicación desarrollada en esta práctica simula una tienda de productos, permitiendo gestionar inventario mediante una interfaz gráfica conectada a una base de datos MySQL. A través de este ejercicio, el estudiante consolida conocimientos sobre programación orientada a objetos, diseño de interfaces gráficas y acceso a datos en aplicaciones reales.

REALIZACIÓN DE PRÁCTICA

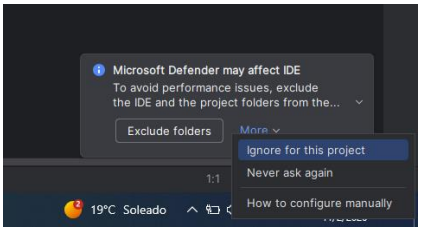
Creamos la base de datos llamada tienda_db

```sql
CREATE DATABASE tienda_db;
USE tienda_db;

CREATE TABLE productos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    precio DECIMAL(10,2) NOT NULL,
    stock INT NOT NULL
);

INSERT INTO productos (nombre, precio, stock) VALUES
('Laptop', 850.00, 10),
('Mouse', 15.50, 50),
('Teclado', 25.00, 30);
```

Vamos a crear nuestro proyecto ‘ArrupeStore’ utilizando JavaFX, con las siguientes configuraciones:



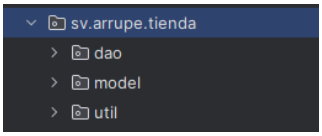
NOTA: No olvides seleccionar la opción ‘ignore for this project’ al momento de crear la aplicación.



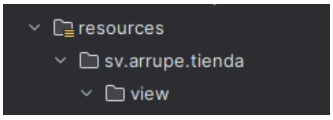
Al crearse el proyecto debemos buscar el archivo de configuración pom.xml para agregar la dependencia de jdbc:

```
<!-- MySQL JDBC -->
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.3.0</version>
</dependency>
```

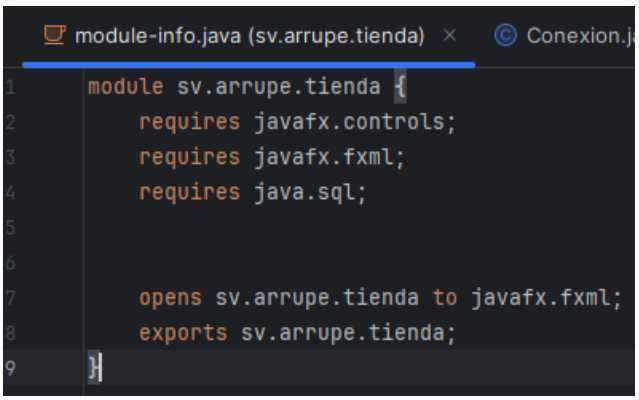
En la estructura del proyecto vamos a encontrar el paquete sv.arrupe.tienda, para ello como trabajaremos con el modelo N capas, debemos crear los siguientes paquetes, que nos permitan gestionar de forma adecuada nuestra aplicación: dao, model, util sin embargo queda pendiente la parte de la vista, JavaFX lo maneja dentro del directorio resources:



En resources vamos a encontrar de nuevo el directorio sv.arrupe.tienda acá es donde vamos a crear la carpeta de las vistas llamado: view



Añadir la librería de jdbc en module-info.java este archivo se encarga de gestionar los módulos a utilizar para el correcto funcionamiento de nuestra aplicación por defecto tiene la configuración de javafx pero aún nos falta el de sql que nos proporciona la dependencia de JDBC



Vamos ahora a trabajar en nuestras clases que nos permitirán ejecutar de forma correcta la aplicación, primero en el paquete util vamos a crear la clase Conexión que se encargara de establecer la comunicación con la base de datos de MySQL:
util/Conexion.java

```java
package sv.arrupe.tienda.util;

import java.sql.Connection;
import java.sql.DriverManager;

public class Conexion {  no usages

    private static final String URL = "jdbc:mysql://localhost:3306/tienda_db";
    private static final String USER = "root";  1 usage
    private static final String PASSWORD = "";  1 usage

    public static Connection getConnection() {  no usages
        try {
            return DriverManager.getConnection(URL, USER, PASSWORD);
        } catch (Exception e) {
            e.printStackTrace();
            return null;
        }
    }
}
```

Ahora vamos a crear el modelo que se va a encargar de inicializar nuestros productos
model/Producto.java

```java
package sv.arrupe.tienda.model;  5 13

public class Producto {

    private int id;
    private String nombre;
    private double precio;
    private int stock;

    public Producto() {}

    public Producto(int id, String nombre, double precio, int stock) {
        this.id = id;
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    public Producto(String nombre, double precio, int stock) {
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    // Getters y Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public double getPrecio() { return precio; }
    public void setPrecio(double precio) { this.precio = precio; }

    public int getStock() { return stock; }
    public void setStock(int stock) { this.stock = stock; }
}
```

En esta ocasión vamos a trabajar con elementos conocidos como data Access object no te preocupes en el módulo 3.4 ahondaremos en el tema, sin embargo en pocas palabras se encargan de establecer el tipo de comunicación con la BDD obteniendo las querys, consultas y subconsultas que interactúan con la bdd:
dao/ProductoDAO

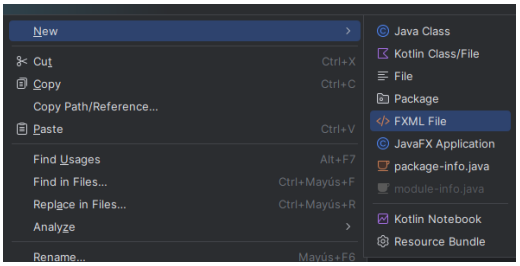
```java
package sv.arrupe.tienda.dao;

import sv.arrupe.tienda.model.Producto;
import sv.arrupe.tienda.util.Conexion;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class ProductoDAO {

    public List<Producto> listar() {
        List<Producto> lista = new ArrayList<>();

        String sql = "SELECT * FROM productos";

        try (Connection conn = Conexion.getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {

            while (rs.next()) {
                Producto p = new Producto(
                        rs.getInt( columnLabel: "id"),
                        rs.getString( columnLabel: "nombre"),
                        rs.getDouble( columnLabel: "precio"),
                        rs.getInt( columnLabel: "stock")
                );
                lista.add(p);
            }

        } catch (Exception e) {
            e.printStackTrace();
        }

        return lista;
    }

    public void insertar(Producto producto) {
        String sql = "INSERT INTO productos (nombre, precio, stock) VALUES (?, ?, ?)";

        try (Connection conn = Conexion.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setString( parameterIndex: 1, producto.getNombre());
            stmt.setDouble( parameterIndex: 2, producto.getPrecio());
            stmt.setInt( parameterIndex: 3, producto.getStock());

            stmt.executeUpdate();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
    public void eliminar(int id) {
        String sql = "DELETE FROM productos WHERE id = ?";

        try (Connection conn = Conexion.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setInt( parameterIndex: 1, id);
            stmt.executeUpdate();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Vamos a crear un controlador principal que se encargara de gestionar toda la comunicación entre las capas del proyecto
MainController.java

```java
package sv.arrupe.tienda;

import sv.arrupe.tienda.dao.ProductoDAO;
import sv.arrupe.tienda.model.Producto;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.*;

public class MainController {

    @FXML private TextField txtNombre;
    @FXML private TextField txtPrecio;
    @FXML private TextField txtStock;
    @FXML private TableView<Producto> tablaProductos;
    @FXML private TableColumn<Producto, Integer> colId;
    @FXML private TableColumn<Producto, String> colNombre;
    @FXML private TableColumn<Producto, Double> colPrecio;
    @FXML private TableColumn<Producto, Integer> colStock;

    private ProductoDAO dao = new ProductoDAO();

    @FXML
    public void initialize() {
        colId.setCellValueFactory( CellDataFeatures<Producto, Integer> data ->
                new javafx.beans.property.SimpleIntegerProperty(data.getValue().getId()).asObject());
        colNombre.setCellValueFactory( CellDataFeatures<Producto, String> data ->
                new javafx.beans.property.SimpleStringProperty(data.getValue().getNombre()));
                new javafx.beans.property.SimpleStringProperty(data.getValue().getNombre()));
        colPrecio.setCellValueFactory( CellDataFeatures<Producto, Double> data ->
                new javafx.beans.property.SimpleDoubleProperty(data.getValue().getPrecio()).asObject());
        colStock.setCellValueFactory( CellDataFeatures<Producto, Integer> data ->
                new javafx.beans.property.SimpleIntegerProperty(data.getValue().getStock()).asObject());

        cargarDatos();
    }

    private void cargarDatos() {
        ObservableList<Producto> lista = FXCollections.observableArrayList(dao.listar());
        tablaProductos.setItems(lista);
    }

    @FXML
    private void agregarProducto() {
        Producto p = new Producto(
                txtNombre.getText(),
                Double.parseDouble(txtPrecio.getText()),
                Integer.parseInt(txtStock.getText())
        );

        dao.insertar(p);
        cargarDatos();
    }
```

```java
    @FXML
    private void eliminarProducto() {
        Producto seleccionado = tablaProductos.getSelectionModel().getSelectedItem();
        if (seleccionado != null) {
            dao.eliminar(seleccionado.getId());
            cargarDatos();
        }
    }
}
```

Ahora vamos a crear la vista, para ello damos clic en la carpeta view creada anteriormente y seleccionamos New, FXML File, llamaremos a nuestra vista como tienda *resources/sv/arrupe/tienda/view/tienda.fxml*

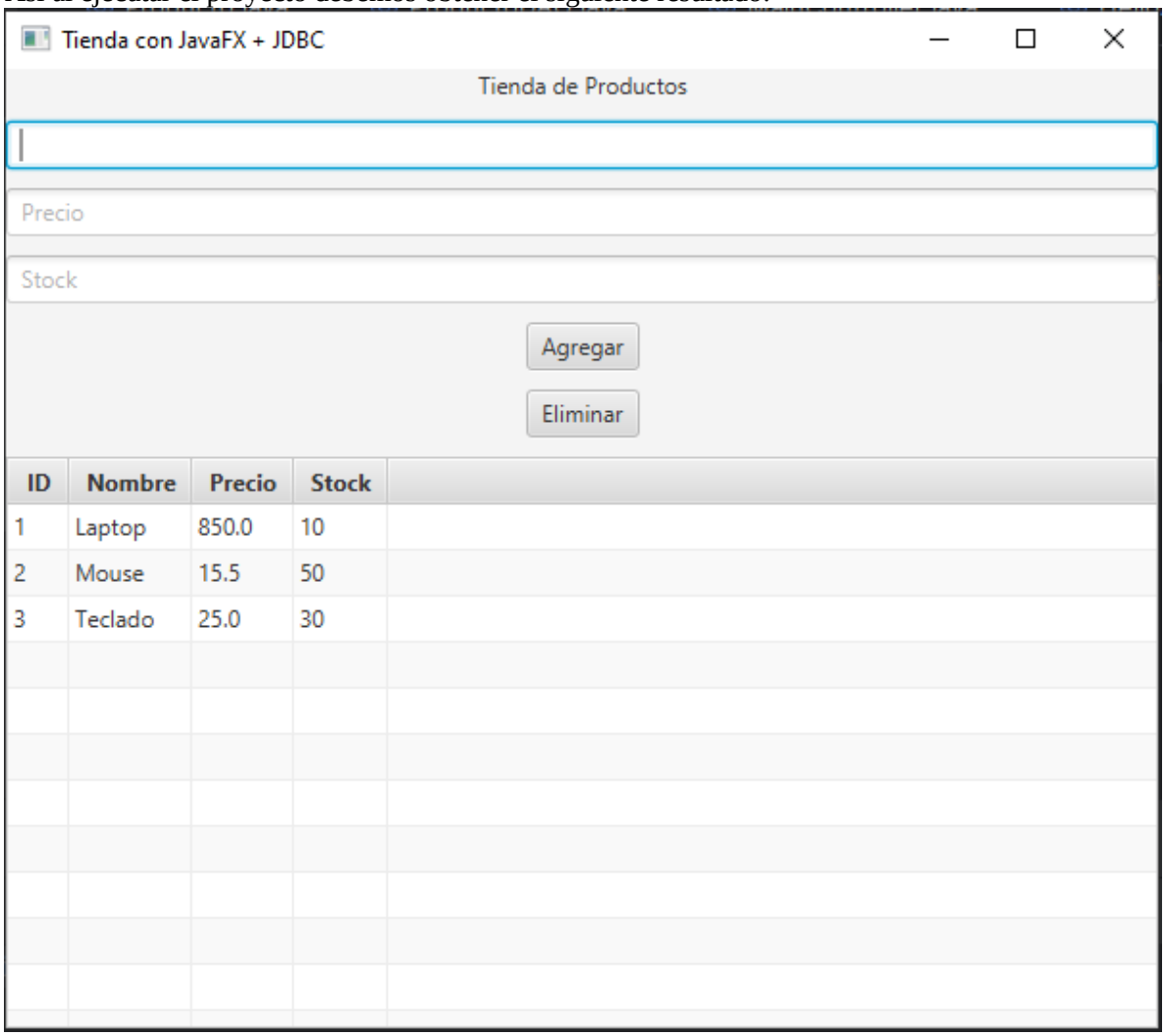



```xml
<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>
<VBox spacing="10" alignment="CENTER"
      xmlns:fx="http://javafx.com/fxml"
      fx:controller="sv.arrupe.tienda.MainController">

    <Label text="Tienda de Productos" />
    <TextField fx:id="txtNombre" promptText="Nombre"/>
    <TextField fx:id="txtPrecio" promptText="Precio"/>
    <TextField fx:id="txtStock" promptText="Stock"/>

    <Button text="Agregar" onAction="#agregarProducto"/>
    <Button text="Eliminar" onAction="#eliminarProducto"/>

    <TableView fx:id="tablaProductos">
        <columns>
            <TableColumn fx:id="colId" text="ID"/>
            <TableColumn fx:id="colNombre" text="Nombre"/>
            <TableColumn fx:id="colPrecio" text="Precio"/>
            <TableColumn fx:id="colStock" text="Stock"/>
        </columns>
    </TableView>

</VBox>
```

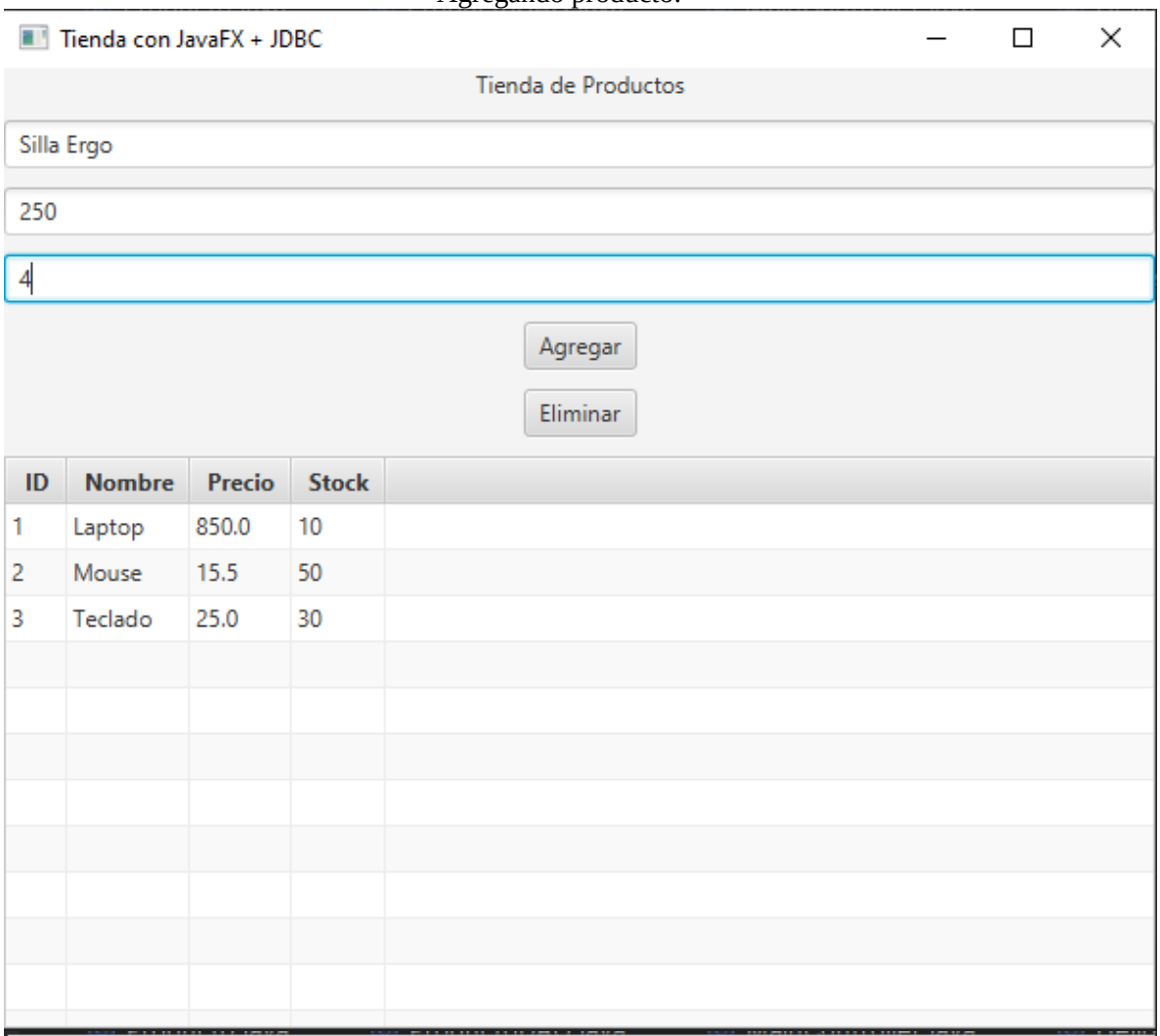
Ahora por ultimo dentro de las clases por defecto vamos a modificar *HelloApplication.java* es quien nos permitirá desplegar la vista tienda.fxml como ventana principal de la aplicación.

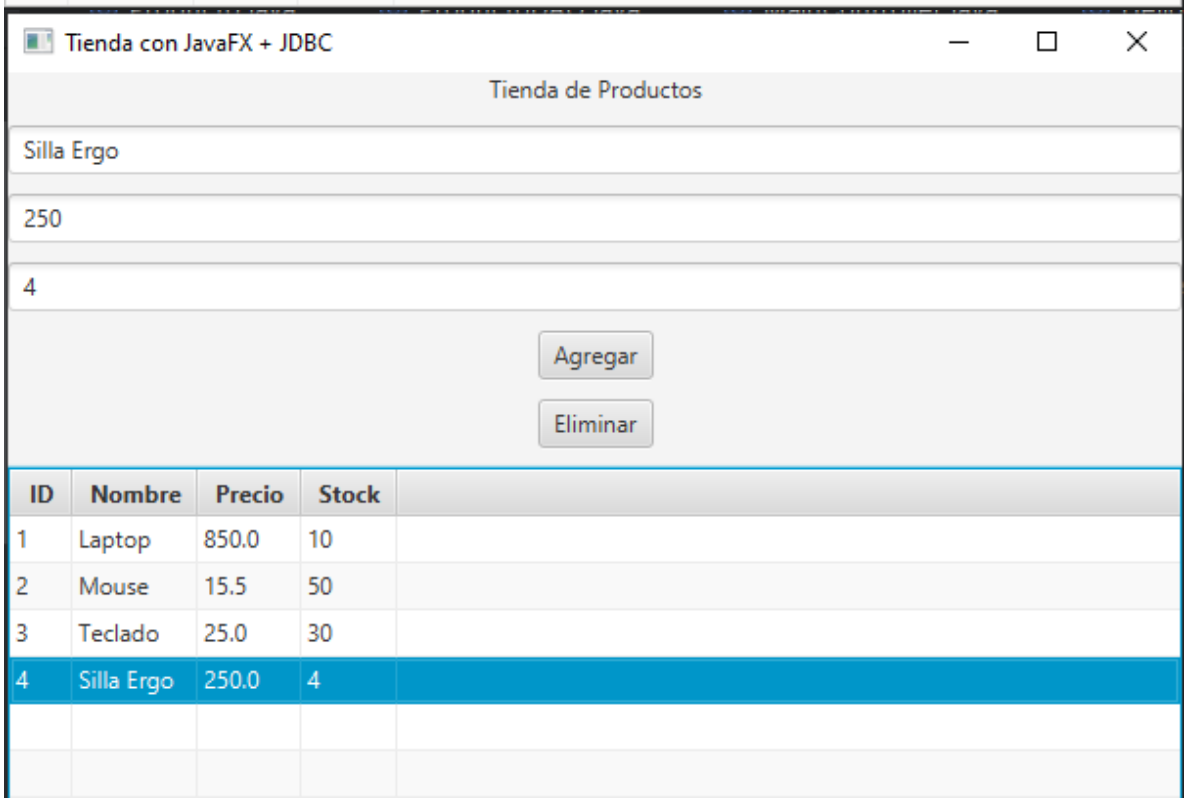
```java
package sv.arrupe.tienda;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.stage.Stage;

public class HelloApplication extends Application {

    @Override
    public void start(Stage stage) throws Exception {

        FXMLLoader fxmlLoader = new FXMLLoader(
                HelloApplication.class.getResource( name: "/sv/arrupe/tienda/view/tienda.fxml")
        );

        Scene scene = new Scene(fxmlLoader.load(),  v: 600,  v1: 500);
        stage.setTitle("Tienda con JavaFX + JDBC");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {  no usages
        launch();
    }
}
```

Así al ejecutar el proyecto debemos obtener el siguiente resultado:



Agregando producto:





ACTIVIDAD COMPLEMENTARIA

1. **Agregar validaciones**

Que el estudiante:

- No permita campos vacíos
- Valide que precio sea numérico
- Valide que stock sea entero positivo
- Muestre Alert si hay error

2. **Agregar botón "Limpiar"**

Después de insertar un producto, limpiar los campos.

3. **Confirmación antes de eliminar**

Mostrar un Alert CONFIRMATION antes de borrar.

4. **Implementar UPDATE (Editar producto)**

Requisitos:

- Seleccionar producto de la tabla
- Cargar datos en los TextField
- Botón "Actualizar"
- Ejecutar:

UPDATE productos SET nombre=?, precio=?, stock=? WHERE id=?

5. **Buscar producto por nombre**

Agregar un campo búsqueda:

SELECT * FROM productos WHERE nombre LIKE ?

6. **Mostrar total de productos y valor total inventario**

Ejemplo:

SELECT SUM(precio * stock) FROM productos;

7. **Aplicar CSS personalizado**

Crear:

styles.css

Cambiar:

- Colores
- Botones
- TableView

- Tipografía

BIBLIOGRAFÍA

Oracle. (2023). *The Java™ tutorials: JDBC basics*. Oracle Corporation. https://docs.oracle.com/javase/tutorial/jdbc/

OpenJFX. (2024). *JavaFX documentation*. https://openjfx.io/

Oracle. (2023). *Java Platform, Standard Edition documentation*. https://docs.oracle.com/en/java/

MySQL. (2024). *MySQL 8.0 reference manual*. Oracle Corporation. https://dev.mysql.com/doc/

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Eckel, B. (2006). *Thinking in Java* (4th ed.). Prentice Hall.

Sosa, D. (2026). Orientaciones prácticas para el desarrollo de aplicaciones JavaFX con JDBC mediante arquitectura MVC. Material didáctico no publicado.